In this unit we make a few assumptions regarding complexity analysis and ADT implementations. Below we provide you with a few more crucial observations to keep in mind.

Complexity Assumptions [Week 1-12]

As you have been mastering algorithmic complexity for a while now, this section outlines a couple of important points to keep in mind when you analyse complexity in practice, especially in the assignments.

Input Size

Remember that the complexity of an algorithm must be measured with respect to the size of the input this algorithm takes. For instance, sorting algorithms deal with finite collections of items and their complexity depends on *how many items* are in the input collection, hence the size of the input is the number of items. When dealing with ADTs, we treat them as collections and hence the rules are similar.

Numeric algorithms though normally take one or more numbers as input and so the complexity of such algorithms must be measured as a function of *size of those numbers*. The size of a number can be seen as the number of digits in its base-k numeral representation. Depending on the base k, for instance binary ($k = 2$) or decimal ($k = 10$), the size of a number n is meant to equal $\log_k n$. Since in Computer Science we are dealing with binary numbers, we conventionally assume a number n is represented in the base-2 numeral system and so its size equals $\log_2 n$, or simply $\log n$.

Do not forget to say explicitly *with respect to what* you analyse the complexity of an algorithm. This will impact your marks.

Cost of Comparison

When we analysed the complexity of the basic sorting algorithms, including Bubble Sort, Selection Sort, and Insertion Sort, we assumed that these algorithms are used to sort collections of *numeric* items, for instance, integer numbers. This assumption allowed us to ignore the *cost of item comparison* in our complexity analysis because arithmetic and comparison operations on numeric inputs of a *fixed size* are constant-time operations.

Our knowledge of basic arithmetic operations from primary school tells us that two integers a and b can be added $a + b = c$ using the **columnar addition** algorithm, in which we spend *linear* time on the number of digits in a and b. In fact, it is linear on the number of digits in $\max(a, b)$. Similarly, assuming n and m are the number of digits in a and b respectively, multiplying $a \cdot b = c$ can be done using the **long multiplication** algorithm, which makes $n \cdot m$ elementary steps (you are invited to check this!). If $n = m$ then the time spent is *quadratic*. Similarly, when comparing two integers a and b of the same size n, we examine *all* the i'th pairs of digits a_i and b_i one by one, s.t. $i \in \{1, \ldots, n\}$, to check if they are the same or not, until we find such i that $a_i \neq b_i$ or all pairs of digits are traversed, which constitutes linear time on n.
The good news is that, as we know, our modern 32-bit or 64-bit computers have a fixed-size representation of integers, namely 32 or 64 bits per integer. This allows us and also a CPU to spend constant time when executing arithmetic instructions on integers - as the input size does not grow at all, we can be sure that columnar addition will take a linear number of steps, i.e. $c \cdot 32$ or $c \cdot 64$. In the case of CPUs, the amount of time spent equals a few dozens *CPU cycles* per operation on integers, which a user may see as performed *in one go*.

However, in general, it is often important to estimate how much time is spent on item comparison and other seemingly cheap operations. For instance, if we are sorting collections of n strings using Selection Sort, we cannot say the complexity is $O(n^2)$ because we have to compare our string items during the operation of Selection Sort and how costly this is depends on how large the strings are. We can surely claim the complexity is still $O(n^2)$ if measured solely on n but this claim would not reflect what happens with the algorithm in practice if we have to deal with strings with millions characters each. In such cases, we must include the cost of comparison into account.

This is conventionally taken into account when talking about the *complexity of ADT operations*. As a general rule of thumb, keep this in mind when reasoning about algorithmic complexity in our unit, including the assignments, but also throughout your study and career. However, if we don't make explicit mention of the length of the values being provided as a separate variable, you don't need to analyse with respect to the size of these values. (For example, a sorting algorithm we don't need to include cost of comparison because it is ambiguous what the input values are. If we said they are strings with max length M, then M should be used in analysis)

Reference complexity of *array-based* Set operations [Week 2-12]

This section gives **reference complexity** of `union()`, `intersection()`, and `difference()` operations on ***array-based sets*** to be used in your analysis, ***whenever applicable in your code and interview questions***.

Recall that this lesson provides an example implementation of the set union operation for array-based Sets. This implementation makes use of the `add()` operation, and its overall complexity is said to be $O(m \cdot (n + m) \cdot c_{=})$, where m and n are the sizes of input sets and $c_{=}$ is the cost of item comparison. Let's look into this again:

<> Python

```python
def union(self, other: ArraySet[T]) -> ArraySet[T]:  # let n = |self|, m = |other|
    res = ArraySet(len(self.array) + len(other.array))  # O(n + m)

    # add all elements of self
    for i in range(len(self)):       # n times
        res.array[i] = self.array[i] # O(1)
    res.size = self.size             # O(1)
    # the complexity of this part is O(n), which is dominated by the next

    # add all elements of other, if they are not yet in
    for j in range(len(other)): # m times
        res.add(other.array[j]) # O(res_size * comp), i.e. O((n + m) * comp)
    # the complexity of this part is O(m * (n + m) * comp)

    return res
```

Although there is nothing wrong with this implementation, its complexity is $O(m \cdot (n + m) \cdot c_{=})$, which is not ideal.

We can do better! Namely, we do not have to use `add()` here, and instead manually check whether the elements of `other` should be added to the result, which will result in a *faster* implementation of the `union()` operation. Here is how it may look like:

<> Python

```python
def union(self, other: ArraySet[T]) -> ArraySet[T]:  # let n = |self|, m = |other|
    res = ArraySet(len(self.array) + len(other.array))  # O(n + m)

    # add all elements of self
    for i in range(len(self)):       # n times
        res.array[i] = self.array[i] # O(1)
    res.size = self.size             # O(1)
    # the complexity of this part is O(n), which is dominated by the next

    # add all elements of other, if they are not in self
    for j in range(len(other)):           # m times
        if other.array[j] not in self:    # O(n * comp)
            res.array[res.size] = other.array[j] # O(1)
            res.size += 1                        # O(1)
    # the complexity of this part is O(m * n * comp)

    return res
```

Observe that this implementation again comprises two major parts: adding the elements of `self` and then adding the elements of `other`. However, since we are not using `add()`, the latter part becomes cheaper and costs only $O(m \cdot n \cdot c_{=})$. We will assume this implementation's complexity of the union operation on array-based Sets to be the reference one.

Similarly, we can devise an implementation of `intersection()` of two array-based sets having the same complexity:

<> Python

```python
def intersection(self, other: ArraySet[T]) -> ArraySet[T]:  # let n = |self|, m = |other|
    res = ArraySet(min(len(self.array), len(other.array)))  # O(min(n, m))

    # add all elements of self
    for i in range(len(self)):           # n times
        if self.array[i] in other:       # O(m * comp)
            res.array[res.size] = self.array[i] # O(1)
            res.size += 1                       # O(1)
    # the complexity is O(n * m * comp)

    return res
```

Similarly, a possible implementation of `difference()` sharing the above complexity may look as follows:

<> Python

```python
def difference(self, other: ArraySet[T]) -> ArraySet[T]:  # let n = |self|, m = |other|
    res = ArraySet(len(self.array))  # O(n)

    # add all elements of self
    for i in range(len(self)):           # n times
        if self.array[i] not in other:   # O(m * comp)
            res.array[res.size] = self.array[i] # O(1)
            res.size += 1                       # O(1)
    # the complexity is O(n * m * comp)

    return res
```

Whenever applicable in your assignment code or in an interview, given two ***array-based sets*** of size n and m, you should assume the **reference complexity** of the operations `union()`, `intersection()`, and `difference()` to be $O(n \cdot m \cdot c_{=})$. Otherwise, your marks will be affected.